Chunking Strategies

Modulo 2 · Lezione 10



I documenti lunghi di Lumen

Tre documenti **veri** di Lumen, più lunghi di quelli visti finora.

`guida-smart-working.txt`

La policy HR: giornate da remoto, la postazione di casa, contributi e regole.

`5.100 caratteri`

`policy-resi-garanzia.txt`

Recesso, rimborsi, garanzia 5+2 anni, l'estensione SRV-03, le eccezioni.

`5.396 caratteri`

`manuale-lumadesk.txt`

Il manuale della LD-200: montaggio, centralina, memorie, codici di errore.

`5.681 caratteri`

Sono **già puliti** (quella fatica l'avete fatta a Modulo 2 · Lezione 5-6) — ma **16.177 caratteri in 3 vettori**? Un testo così non sta sensatamente in un embedding solo. Vediamo perché.



Agenda

Un solo file per tutta la serata: `working/chunk.py`.

	Blocco	Dove si lavora	Tempo
	Teoria: dove cade il chunking, le tre strategie, i costi	slide	32′
	Live coding: `chunk_fixed` e `chunk_recursive` (le scrivete) + demo `chunk_semantic` (si guarda)	`working/chunk.py`	45′
	Esperimenti: le manopole, le tre query, la tabella	`working/chunk.py`	38′
	Wrap-up: reveal + ponte alla prossima lezione	slide	5′

= scrive il docente, voi replicate (o guardate) = scrivete voi, docente in affiancamento

> Chiave OpenAI: quella di **Modulo 2 · Lezione 1-9** (persa? alzate la mano). La collection si **ricrea da zero** a ogni giro: cambi un'impostazione, rifai l'indice.



1. Perché si spezza



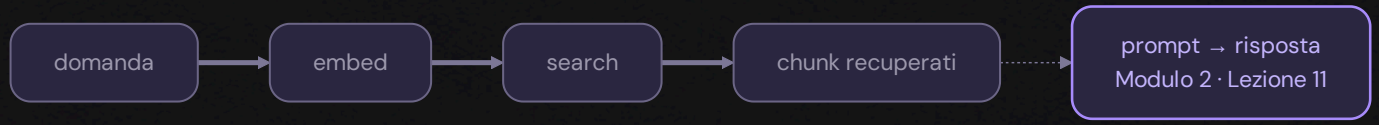
L’ordine di esecuzione

I pezzi costruiti finora girano in due momenti diversi. Il chunking sta nel primo:

INDICIZZAZIONE — UNA VOLTA SOLA, OFFLINE



INTERROGAZIONE — A OGNI DOMANDA



> Il chunking gira **una volta sola**, prima di tutto il resto: da lì in poi si lavora sui pezzi che avete deciso.

Le query sui documenti interi

Indicizziamo i 3 documenti così come sono — un vettore l'uno — e facciamo le tre domande della serata:

Query	Documento in testa	Score	E il 2°?
«Quanti giorni per il reso comprato nei saldi?»	`policy-resi-garanzia`	**0.466**	0.340
«Quanto vale il contributo postazione, entro quando?»	`guida-smart-working`	**0.421**	0.328
«La centralina mostra E07: cosa devo fare?»	`manuale-lumadesk`	**0.380**	0.330

Score piatti

Il vettore di un documento intero è la **media di dieci argomenti**: non somiglia davvero a nessuna domanda. I secondi posti stanno a un soffio — il metro **non discrimina**.

«Risultato» = 5.000+ caratteri

Il documento giusto **non è la risposta**: è un compito a casa («cercatela voi lì dentro»). Questi risultati andranno **dentro un prompt** nella prossima lezione.

> Trovare il **documento** giusto non basta: il retrieval deve trovare il **pezzo** giusto.



Le due ragioni (e una definizione)

Due problemi diversi, una sola causa: il documento intero è l'unità **sbagliata**.

Context window (e conto in token)

Nella prossima lezione i risultati finiscono **dentro un prompt**: un documento intero non ci sta — e quando ci sta, lo **paghi tutto** (Modulo 2 · Lezione 2) per usarne tre righe.

Granularità del retrieval

Un embedding per 10 argomenti = significato **annacquato**: la media di tutto non somiglia a niente. L'avete appena visto: score piatti, secondi posti attaccati.

Il chunk è l'unità del retrieval

Si spezza **prima** di embeddare · si indicizza il **pezzo** · si recupera il **pezzo**. Tutta la domanda della serata è: **DOVE si taglia?**

> E il non-chunking? È solo un chunking con **size infinito** — infatti stasera si prova con la stessa manopola.



2. Le tre strategie



Fixed-size: ogni N caratteri, taglio

La più semplice: blocchi di `size` caratteri — anche a metà parola, anche a metà frase. E l'**overlap**: ogni blocco riparte un po' indietro.



Pro: prevedibile

Sai **quanti** chunk escono e di che **taglia** — sempre, su qualsiasi testo.

L'overlap è una rete di sicurezza COMPRATA

Ogni confine esiste anche ricucito — ma ogni carattere ripetuto è un **token ripagato**. Il conto tra poco.

Recursive: il taglio rispetta il testo

Si prova il separatore più forte e si *ricorre* ai più deboli solo se serve: il taglio cade sull'**ultimo separatore che ci sta** nel size, poi si riparte da lì.

`"\n\n"` paragrafo → `"\n"` riga → `". "` frase → `taglio secco` solo se nient'altro ci sta

Pro: mai una parola rotta

I tagli cadono su titoli, paragrafi e frasi: chunk che **sembrano scritti apposta**.

Contro: si fida della FORMA

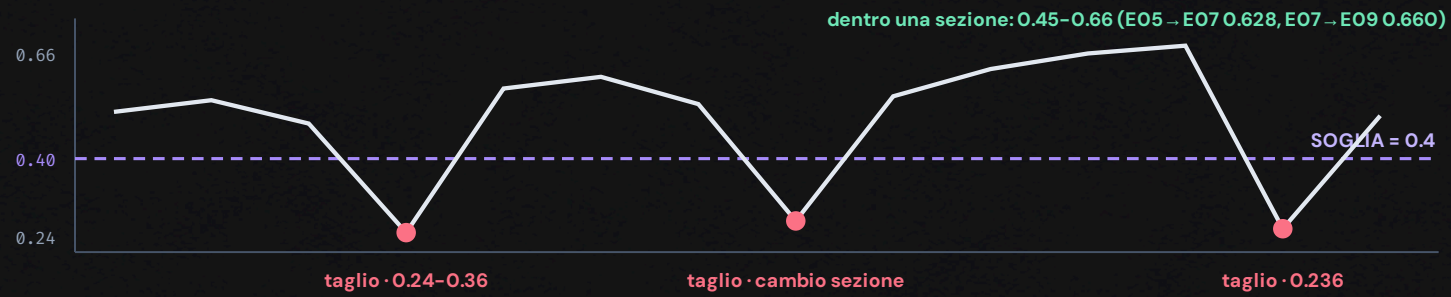
Se chi scrive va a capo **a metà ragionamento**, il taglio lo segue nel burrone. Il testo non sempre va a capo dove cambia il discorso.

> Nei framework la incontrerete come *RecursiveCharacterTextSplitter* — stasera la scrivete in **dieci righe**.



Semantic: il taglio segue il discorso

Gli embedding sono strumenti vostri da Modulo 2 · Lezione 7: embedding di ogni **frase**, similarità tra frasi **adiacenti** — dove crolla, il discorso è cambiato → taglio. Sul manuale:



I tagli si trovano da soli

Con soglia **0.4** i tagli cadono sui **confini di sezione** — nessuno gli ha detto dove sono i titoli.

Si paga due volte

Prima le **frasi** (per decidere i tagli), poi i **chunk** (per l'indice): il chunking stesso va a consumo.

Qui la manopola è la **SOGLIA**: alta = tagli ovunque, bassa = chunk lunghi.

La stessa pagina, spezzata nei tre modi

La sezione «saldi» della policy, tre geometrie. Il retrieval le vedrà **diversissime**:

fixed 500/0

…rimborso proporzionale ai prodotti rientrati

PRODOTTI IN PROMOZIONE E SALDI · …la finestra di reso

scende a 14 giorni… buono acquisto…

blocchi identici, frase **spezzata**

recursive 500

RIMBORSI · il rimborso parte quando…

PRODOTTI IN PROMOZIONE E SALDI · … scende a 14 giorni… buono acquisto

GARANZIA LEGALE E COMMERCIALE · …

tagli sui separatori, frase **intera**

semantic 0.4

…tutto il blocco «rimborsi», insieme

il blocco «saldi», dal titolo alla fine

il blocco «garanzia», insieme

un chunk = **un argomento**

> Sull'intero corpus: **34** · **45** · **77** chunk — stessa pagina, tre geometrie.



Quanto costa

Il conto dell'indicizzazione dei 3 documenti (16.177 caratteri), in tre numeri:

= 0%

Spezzare è gratis

Interi **4.804** token · fixed 500/0 **4.831** · recursive **4.765**. Stessi caratteri, stesso conto: si paga il testo, non il numero di pezzi.

+42%

L'overlap si paga

Con 500/150: **6.853** token contro 4.831. Ogni carattere ripetuto è un token ripagato — la rete di sicurezza ha un prezzo.

×2

Semantic raddoppia

9.389 token: prima si embeddano tutte le frasi per decidere dove tagliare, poi i chunk per l'indice.

> Qui sono frazioni di centesimo. A milioni di documenti questo rapporto **è il budget**.



L’ambiente

Quattro righe e siete operativi (passo-passo anche in `scripts/CONSEGNA.md`):

```
cd scripts
python3 -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env      # dentro: la STESSA chiave OpenAI di Modulo 2 · Lezione 1-9
```

Niente corpus da copiare stasera

I documenti sono **in chiaro** in `dataset/` : gli embedding si calcolano **a ogni giro** e la collection `lumen_chunks` si **ricrea da zero** — è il punto della serata.

aikit/ è cresciuto di un modulo

`vectorstore.py` — il **VOSTRO**, scritto nella lezione precedente, promosso nel toolkit. Si importa, non si riscrive.



Live coding

Hint: osservate e replicate

`working/chunk.py` · 45 minuti (i primi 5: giro del file insieme)

si torna alle slide
dopo la demo

`TODO 1 · ~15' · si replica`

`chunk_fixed`

Un `while`, una fetta di stringa, l'inizio che avanza di `size - overlap`.

`TODO 2 · ~15' · si replica`

`chunk_recursive`

Lo stesso `while`, con dentro il ciclo sui tre separatori: `rfind`, taglio, si riparte da lì. Per lanciarla: `STRATEGIA = "recursive"`.

`DEMO · ~10' · si guarda`

`chunk_semantic`

È **già nel file**: la leggiamo insieme e la lancio io. Voi la **userete** negli esperimenti — capirla sì, riscriverla no.

Dopo ogni funzione: si rilancia lo script

Ogni lancio rifà il giro intero sui tre documenti — chunk, indicizzazione, le tre query — e stampa **i tagli** (dove cadono, quanti chunk, di che taglia) e il primo risultato **per intero**. Sono già i primi dati della vostra tabella.



Lo scheletro delle due funzioni

La forma, non la soluzione. Questa slide resta qui mentre scrivete:

```
chunk_fixed(testo, size, overlap)

pezzi = lista vuota
inizio = 0
finché inizio è dentro il testo:
    aggiungi la fetta da inizio, lunga size
    inizio avanza di size - overlap
restituisci pezzi
```

```
chunk_recursive(testo, size)

pezzi = lista vuota
inizio = 0
finché resta testo:
    se il resto ci sta tutto → è l'ultimo pezzo
    guarda la finestra di size caratteri
    per separatore in \n\n, \n, ". ":
        posizione = rfind → il primo che ci sta vince
    taglia lì, inizio riparte dopo il separatore
    (nessun separatore → taglio secco a size)
```

> I nomi delle variabili sono questi: se vi perdete tra il vostro editor e il mio schermo, guardate qui.



Recap: le tre strategie

`AL RIENTRO` Quello che avete appena visto:

fixed taglia alla cieca

A 500/0 le tre query tornano **tutte a metà**: il chunk dei saldi si interrompe su «…la finestra di reso» — il numero è nel chunk dopo.

recursive rispetta i separatori

Saldi **0.695** ed E07 **0.735** tornano **interi in testa**… ma il CONTRIBUTO resta a metà anche coi tagli educati.

semantic si orienta da sola

Tagli sui **confini di sezione** senza conoscerli — e a schermo la riga `[openai]` è passata **due volte**: si paga.

Perché il contributo resiste?

Lo scoprite negli **esperimenti**: lo script stampa i tagli anche sulla `guida-smart-working` — guardate **dove finisce un chunk e dove comincia il successivo**.



Esperimenti

TOCCA A VOI — muovete le manopole

`working/chunk.py` · 38 minuti · si toccano SOLO le 4 manopole in testa al file

si torna alle slide
al mio segnale, per il reveal

Il giro

Cambia la config → rilancia → confronta la **risposta attesa** (la stampa lo script) col **1° risultato per intero** → annota una riga nella **tabella in fondo al file**. Una manopola per volta, quando potete.

`da provare`

`500/0` · `overlap 150` · `size 1000` · `300/100` · `recursive 500` · `semantic 0.4 e 0.5`

Le tre viste girare nel live coding (500/0, recursive 500, semantic 0.4): annotatele e passate alle altre.

Domanda guida: quale config porta a casa **TUTTE e tre** le query? Ce n'è una sola? (`OVERLAP` sempre minore di `CHUNK_SIZE` .)

OK
La risposta è lì, tutta.

META
C'è solo un pezzo: si interrompe a metà.

FUORI
Non è la risposta: parla d'altro, o ve la fa cercare.

Completato quando:
Avete provato **tutte e tre le strategie**, una config è **OK su tutte e tre** le query, e sapete dire **quale manopola** ha fatto la differenza.



La tabella completa

AL RIENTRO Compilata in preparazione — le vostre dovrebbero somigliarle:

Config	chunk	saldi	contributo	E07
documenti interi	3	FUORI	FUORI	FUORI
fixed 500/0	34	META	META	META
fixed 500/150	48	OK 0.684	META *	META
fixed 1000/0	18	META	FUORI	FUORI
fixed 300/100	82	OK 0.646	META	META
recursive 500	45	OK 0.695	META	OK 0.735
semantic 0.4	**77**	**OK 0.700**	**OK 0.527**	**OK 0.665**
semantic 0.5	119	OK	META	OK

(*) a 500/150 il chunk ricucito dall'overlap **ESISTE** (importo E scadenza insieme) ma resta **terzo** — l'overlap ricuce il testo, non garantisce il ranking. Il contributo era **a cavallo di due paragrafi**: recursive taglia proprio lì; semantic vede che il discorso **continua** (le frasi si somigliano) e non taglia.



Si sceglie sperimentando

Il metodo, prima ancora del codice:

01

Nessuna strategia domina

Stasera **ognuna ha vinto e perso** da qualche parte: fixed la più economica, recursive la più pulita, semantic la più precisa (e cara).

02

La config giusta esiste — QUI

semantic 0.4 vince **su questo corpus e queste domande**. Su contratti, codice o chat la tabella si **rifà da capo**.

03

Sempre lo stesso giro

Cambia una manopola → rilancia → leggi → annota. Vale per il chunking come per **tutto il tuning** che arriverà.

Il chunking è la leva più a monte

Tutto quello che viene dopo — ricerca, prompt, risposta — **eredita i vostri tagli**. Un taglio sbagliato non si recupera a valle.

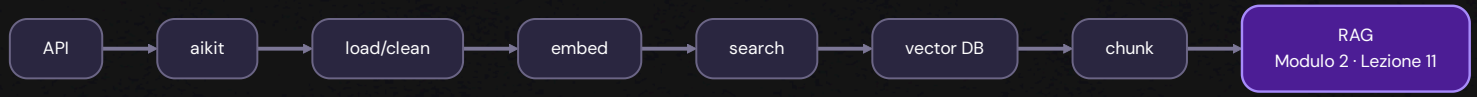


Verso il primo RAG completo

Ultimo passo, adesso: rimettete la config che avete scelto e rilanciate — su disco resta la collection dell'**ultimo lancio**. In dubbio: `recursive`, size 500.

Da lì riparte la prossima lezione:

L'ORDINE IN CUI L'ABBIAMO IMPARATO



query → search sulla collection → i chunk DENTRO un prompt → risposta con citazioni

Il primo **RAG completo** del corso: l'LLM risponde *citando i documenti di Lumen*.

Grazie

Prossima lezione (**Modulo 2 · Lezione 11**): Prompt Engineering Avanzato.

